

LAB 02 - Classes, Objects, Data Members & Member Functions

BSAI Program (BAI Section) | Fall 2026 | 3 Hours

Instructor	Muhammad Shahid Bhatti	Designation	Assistant Professor
Department	Computer Science	Office	H-17
Lab basis	Lectures 3 and 4	Execution mode	Windows Command Prompt
Expected submission	Lab02 folder pushed to GitHub	Individual work	Yes

Lab focus: turn the concepts from Classes & Objects and Data Members & Member Functions into working Java programs. Students must predict, code, compile, run, and explain rather than only copy code. Each task now ends with a short concept check that must be answered before the instructor signs off the task.

Scope discipline: Do not introduce constructors, access modifiers/encapsulation, static members as a concept, inheritance, arrays/collections, or exception handling in this lab. The only required static member is the standard main method boilerplate.

1. Learning Outcomes

- Create Java classes and instantiate multiple objects using new Student() without defining constructors yet.
- Declare instance variables and demonstrate that different objects maintain independent state.
- Write instance methods that modify state, accept parameters, use local variables, and return values.
- Apply Java naming conventions to classes, fields, variables, parameters, and methods.
- Create and call valid overloaded methods and identify invalid overloads using method signatures.
- Demonstrate Java pass-by-value for primitives and for object-reference values.
- Compile and run multi-file Java programs from Windows Command Prompt and interpret compiler feedback.

2. Lab Rules and Working Folder

- Compile and run all programs from Windows Command Prompt. Do not use the IDE Run button for this lab.
- You may use a text editor/IDE only for editing if permitted by the instructor; compilation evidence must come from javac and java commands.
- Work individually. You may discuss concepts, but your code, predictions, explanations, and Git commits must be your own.
- Do not copy final code from another student. Instructor may ask you to modify your program live to verify understanding.
- Use meaningful Java names. Avoid class names such as Test1, ABC, xyz, or random abbreviations.
- Before running each task, write your expected output in a comment or a small text file named predictions.txt.

```
cd C:\CSC241\CIIT-SP26-BAI-XXX-LHR
mkdir Lab02
cd Lab02
java -version
javac -version
git --version
```

Recommended folder: C:\CSC241\CIIT-SP26-BAI-XXX-LHR\Lab02

3. Three-Hour Activity Plan

Time	Activity	Evidence / goal
0-15 min	Setup + instructor checkpoint	Verify JDK/Git, create Lab02 folder, review compile/run commands.
15-40 min	Task 1 - Multiple objects	Class vs. object, references, identity, independent instance state.
40-70 min	Task 2 - Methods and returns	Instance methods, parameters, local variables, return types.
70-95 min	Task 3 - Method overloading	Signatures, argument count/types, valid and invalid overloads.
95-130 min	Task 4 - Pass-by-value experiments	Primitive copy, copied reference, mutation vs. reassignment.
130-165 min	Task 5 - Integrated BAI challenge	Design and implement a small AIExperiment class from requirements.
165-175 min	GitHub checkpoint	Commit and push Lab02 to existing repository.
175-180 min	Exit check	Instructor verification + short reflection.

4. Task 1 - Build a Class and Prove Independent Object State

Task	Three Student objects from one Student class
Time	25 minutes
Concept focus	Class anatomy, object instantiation, reference variables, identity/state, instance variables.
Measurable evidence	Student.java + Task1Demo.java; console output showing three independent objects and a before/after change to only one object.

Completion checks

☐	Create Student with fields studentId, name, completedCredits.
☐	Create exactly three Student objects.
☐	Use BAI example data such as Abeer Amina, Ali Ishtiaq, and Abdul Rehman Azam, or your own class data.
☐	Print all three objects before any change.
☐	Increase completedCredits of only one object and print all three again.
☐	Add a two-line comment explaining why the other two objects did not change.

```
class Student {
    String studentId;
    String name;
    int completedCredits;
}

// In Task1Demo.main(...):
Student s1 = new Student();
Student s2 = new Student();
Student s3 = new Student();
```

```
// Assign values, print all objects, modify only one object, print again.
```

Instructor check: Ask the student to change `s2.completedCredits` live and explain which object changes and why.

Task 1 Concept Check - Classes, Objects, and Instance State

Answer briefly in 1-2 sentences before asking the instructor for the completion check.

1. What is the difference between the Student class and the objects `s1`, `s2`, and `s3` created from it?

Answer: _____

2. Why does changing `s2.completedCredits` not change the `completedCredits` value stored in `s1` or `s3`?

Answer: _____

3. Which variables in Student represent persistent object state, and why are they instance variables rather than local variables?

Answer: _____

5. Task 2 - Add Behavior: Parameters, Local Variables, and Return Values

Task	Extend Student with meaningful member functions
Time	30 minutes
Concept focus	Instance methods, receiver object, parameters vs. arguments, local variables, void and non-void return types.
Measurable evidence	Revised Student.java + Task2Demo.java; output proves state modification and returned results.

Completion checks

☐	Add void <code>addCredits(int credits)</code> that changes <code>completedCredits</code> .
☐	Add int <code>remainingCredits(int degreeCredits)</code> using a local variable named <code>remaining</code> .
☐	Add String <code>summary()</code> returning one readable line containing ID, name, and credits.
☐	Call all three methods on at least two different Student objects.
☐	Write the expected result before each call in <code>predictions.txt</code> .
☐	Explain which variable is a field, which is a parameter, and which is local.

```
// Required method shapes - write the bodies yourself
void addCredits(int credits) { ... }

int remainingCredits(int degreeCredits) {
    int remaining = ...;
    return remaining;
}

String summary() { ... }
```

Task 2 Concept Check - Methods, Parameters, Locals, and Returns

Answer briefly in 1-2 sentences before asking the instructor for the completion check.

1. In `addCredits(int credits)`, what is the difference between the parameter `credits` and the object field `completedCredits`?

Answer: _____

2. Why is `remaining` inside `remainingCredits(...)` better as a local variable than as an instance field?

Answer: _____

3. What information does the return type of remainingCredits(int degreeCredits) give to the caller?

Answer: _____

6. Task 3 - Method Signatures and Overloading

Task	Create and test overloaded methods
Time	25 minutes
Concept focus	Method signatures, parameter count/type/order, compile-time overload resolution, return type not part of the signature.
Measurable evidence	OverloadDemo.java containing three valid overloads, one deliberately invalid attempt (commented after observing error), and predictions for five calls.

Completion checks

☐	Create enroll(String courseCode).
☐	Create enroll(String courseCode, int section).
☐	Create enroll(int numericCourseCode).
☐	Write the three method signatures as comments above the methods.
☐	Test at least three valid calls and two invalid calls.
☐	Attempt an invalid overload that differs only by return type; capture the compiler message in compiler_notes.txt, then comment out/fix the invalid code.
☐	Explain in one sentence why the return type cannot distinguish overloads.

```
void enroll(String courseCode) { ... }
void enroll(String courseCode, int section) { ... }
void enroll(int numericCourseCode) { ... }

// Predict before compiling:
enroll("CSC241");
enroll("CSC241", 2);
enroll(241);
// enroll();
// enroll("241", "2");
```

Stretch question: If show(long) and show(double) exist, what would show(10) select? Predict first, then verify. This is optional and should not replace the core task.

Task 3 Concept Check - Method Signatures and Overloading

Answer briefly in 1-2 sentences before asking the instructor for the completion check.

1. What is the method signature of enroll(String courseCode, int section)?

Answer: _____

2. Why are enroll(String) and enroll(int) valid overloads, while two methods with the same name and parameter types but different return types are not?

Answer: _____

3. For the call enroll("CSC241", 2), which overload should the compiler select, and what information does it use to make that decision?

Answer: _____

7. Task 4 - Prove Java Pass-by-Value

Task	Three controlled experiments
Time	35 minutes
Concept focus	Primitive values are copied; object reference values are also copied; object mutation differs from parameter reassignment.
Measurable evidence	PassByValueDemo.java + predictions.txt containing predictions and explanations for all three experiments.

Completion checks

☐	Experiment A: pass an int to a method; assign a new value to the parameter; prove caller variable is unchanged.
☐	Experiment B: pass a Student reference; change st.completedCredits; prove caller observes the object mutation.
☐	Experiment C: inside the method assign st = new Student(); prove caller still refers to the original object.
☐	For each experiment write Before, Inside Method, and After values.
☐	Use the exact phrase “the argument value is copied into the parameter” in your explanation.
☐	Avoid the incorrect statement “Java passes objects by reference.”

```
// Keep only main static as normal Java boilerplate.
// You may create a helper object and call instance methods on it.

void changeNumber(int x) {
    x = 99;
}

void changeStudent(Student st) {
    st.completedCredits = 99;
}

void replaceStudent(Student st) {
    st = new Student();
    st.name = "Temporary";
}
```

Required explanation: Why can changeStudent(...) affect the object, while replaceStudent(...) cannot redirect the caller variable?

Task 4 Concept Check - Java Pass-by-Value

Answer briefly in 1-2 sentences before asking the instructor for the completion check.

1. After changeNumber(int x) assigns x = 99, why does the caller's original int variable keep its old value?

Answer: _____

2. If a Student reference is passed to a method, why can changing st.completedCredits be visible to the caller?

Answer: _____

3. Why does assigning st = new Student() inside a method not make the caller's variable refer to the new Student object?

Answer: _____

8. Task 5 - Integrated BAI Challenge: AIExperiment

Scenario: A BAI student is tracking progress of small AI experiments. Design a class that stores experiment state and provides methods to update and query that state. Use only concepts covered through Lecture 4.

ID	Requirement	Evidence
R1	Create class AIExperiment with fields experimentName, completedEpochs, targetEpochs.	☐ Verified
R2	Create at least two independent AIExperiment objects using new AIExperiment(). Assign field values explicitly.	☐ Verified
R3	Add void runEpochs(int epochs) to increase completedEpochs.	☐ Verified
R4	Overload runEpochs(int epochs, int bonusEpochs).	☐ Verified
R5	Add int remainingEpochs() that returns targetEpochs - completedEpochs.	☐ Verified
R6	Add String status() that returns a readable one-line summary.	☐ Verified
R7	Use a local variable inside at least one method.	☐ Verified
R8	Demonstrate that changing one experiment does not change the other.	☐ Verified
R9	Pass one AIExperiment object to a helper method that modifies completedEpochs; explain why the mutation is visible.	☐ Verified
R10	Use Java naming conventions consistently and write predictions before running.	☐ Verified

Minimum output evidence: show initial state of both experiments, state after method calls, remaining epochs, status text, and the pass-by-value mutation experiment.

Do not add yet: constructors, private fields, getters/setters, inheritance, arrays/ArrayList, interfaces, exceptions, file I/O, or static class members beyond main.

Task 5 Concept Check - Integrated Class Design

Answer briefly in 1-2 sentences before asking the instructor for the completion check.

1. In AIExperiment, which data should be stored as instance variables, and which calculations should remain local to methods?

Answer: _____

2. Identify one method in your AIExperiment class that changes object state and one method that only computes or returns information.

Answer: _____

3. If you created overloaded methods in this task, state their signatures and explain why the compiler can distinguish them.

Answer: _____

9. GitHub Checkpoint and Submission Evidence

- Use the same GitHub repository created in Lab 01 unless the instructor gives a different repository policy.
- Create a Lab02 folder in that repository and include only source/evidence files for this lab.
- Do not upload .class files. If your repository uses .gitignore from Lab 01, confirm that *.class remains ignored.
- Use at least two meaningful commits so progress is visible.

```
git status
git add Lab02
git commit -m "CSC241 Lab02 tasks 1-4"

git add Lab02
git commit -m "Complete Lab02 integrated AIExperiment task"
```

```
git push
git status
```

☐	Lab02 contains Student.java
☐	Lab02 contains Task1Demo.java
☐	Lab02 contains Task2Demo.java
☐	Lab02 contains OverloadDemo.java
☐	Lab02 contains PassByValueDemo.java
☐	Lab02 contains AIExperiment.java
☐	Lab02 contains AIExperimentDemo.java
☐	Lab02 contains predictions.txt
☐	Lab02 contains compiler_notes.txt (if used)

Repository link: Submit through the instructor-provided Google Form or course mechanism when announced.

10. Suggested Evaluation Rubric (10 Marks)

Component	Marks	Measurable criterion
Task 1 - class/objects/independent state	1.5	Three objects + correct state + live explanation
Task 2 - methods/parameters/returns	1.5	Required methods, local variable, correct returned values
Task 3 - overloading/signatures	1.5	Three overloads, valid/invalid call reasoning, return-type rule
Task 4 - pass-by-value	2.0	Primitive, mutation, reassignment experiments + correct explanation
Task 5 - integrated AIExperiment	2.5	All core requirements implemented and explained
Command Prompt + Git evidence	0.5	javac/java execution and meaningful commits
Code quality / naming / predictions	0.5	Readable names, clean formatting, predictions before execution

11. Five-Minute Exit Check

1. What is the difference between a class and an object?

Answer: _____

2. Why does changing s1.completedCredits not change s2.completedCredits?

Answer: _____

3. What is the method signature of enroll(String, int)?

Answer: _____

4. Why can two methods not be overloaded by return type alone?

Answer: _____

5. Why can a method mutate an object through a copied reference value but cannot reassign the caller's reference variable?

Answer: _____

Final success criterion: A student who completes this lab should be able to create a class, instantiate independent objects, add meaningful behavior, reason about overloaded calls, and correctly explain Java pass-by-value using both primitives and object references.